



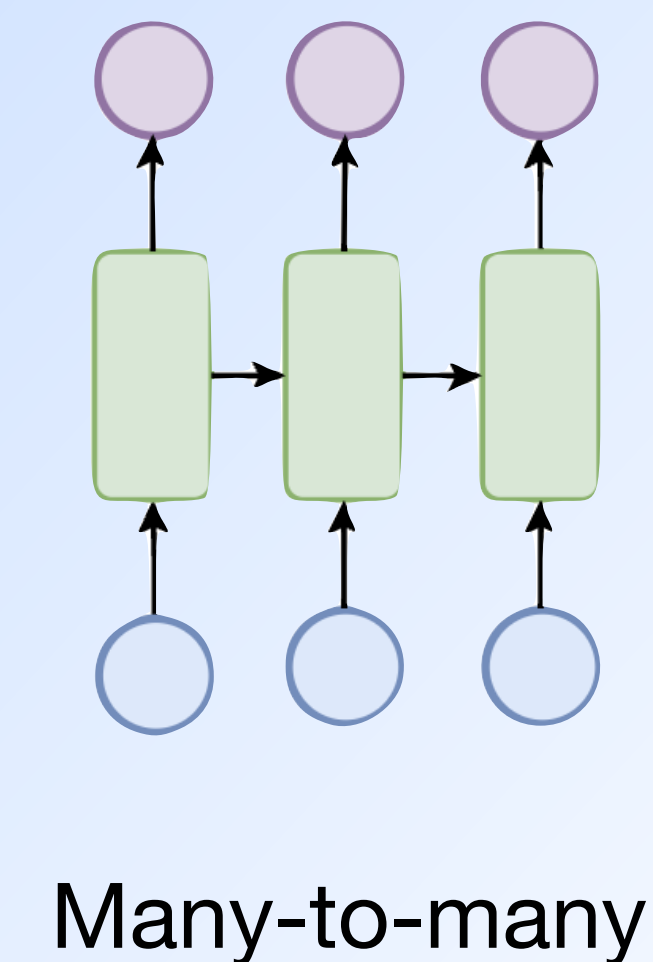
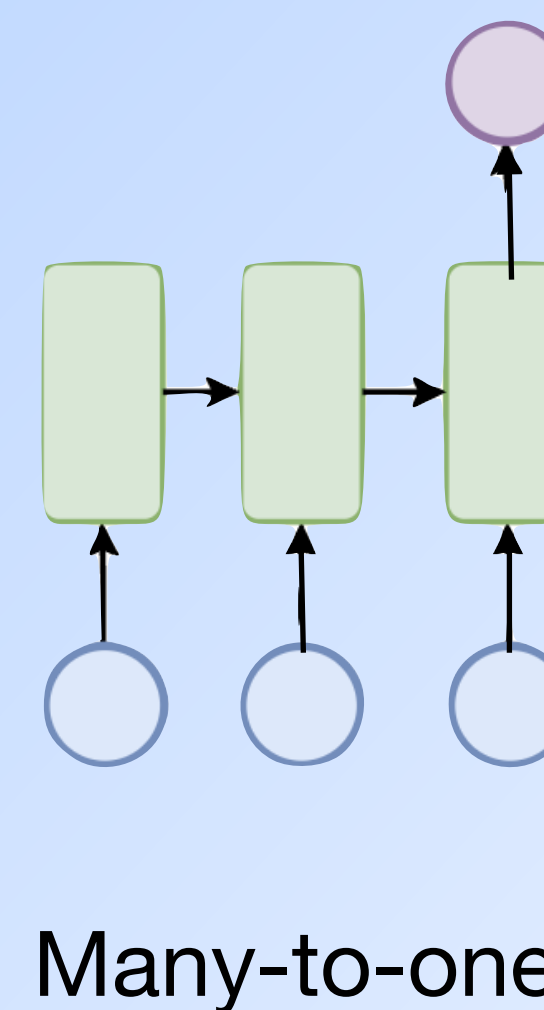
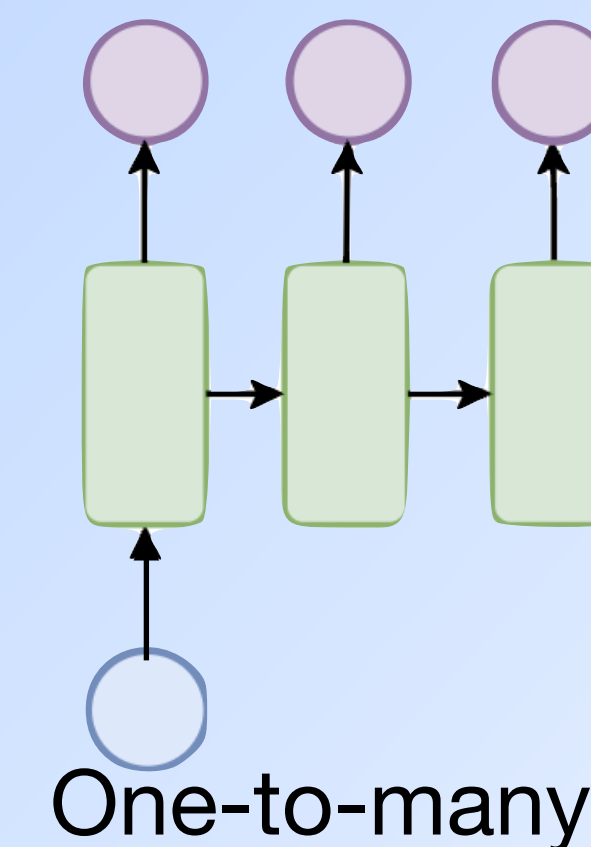
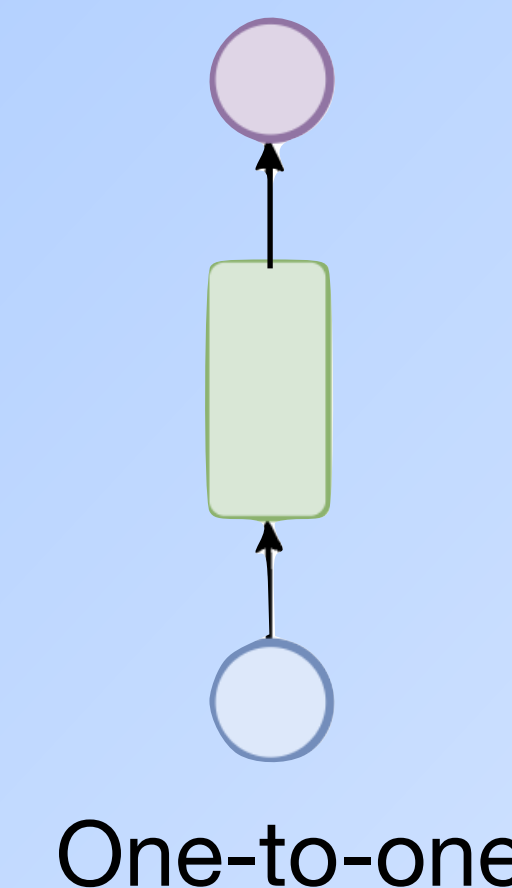
AI61002: Deep Learning Foundations and Applications

Module 6: Recurrent Neural Networks

Dr. Somdyuti Paul

Recurrent Neural Networks

- Recurrent Neural Networks (RNNs) constitute a family of neural networks that enable processing of sequential data.
- RNNs can process inputs as sequences of variable length, and produce variable length outputs as well .
- Examples of sequential learning tasks:
 - One-to-many-image captioning, music generation
 - many-to-one - sentiment analysis, video activity recognition
what emotion the lines of txt contains
 - Many-to-many - text translation, named entity recognition.
given a set of vocabulary ,we have to identify what persons , what places , what organizations ,what actions etc. are named in the txt



Recurrent Neural Networks

- The key idea of RNNs is to store past information in an internal memory, which is updated after processing each element of the sequence.
- In general, the input to a RNN can be represented by a finite length sequence of vectors:

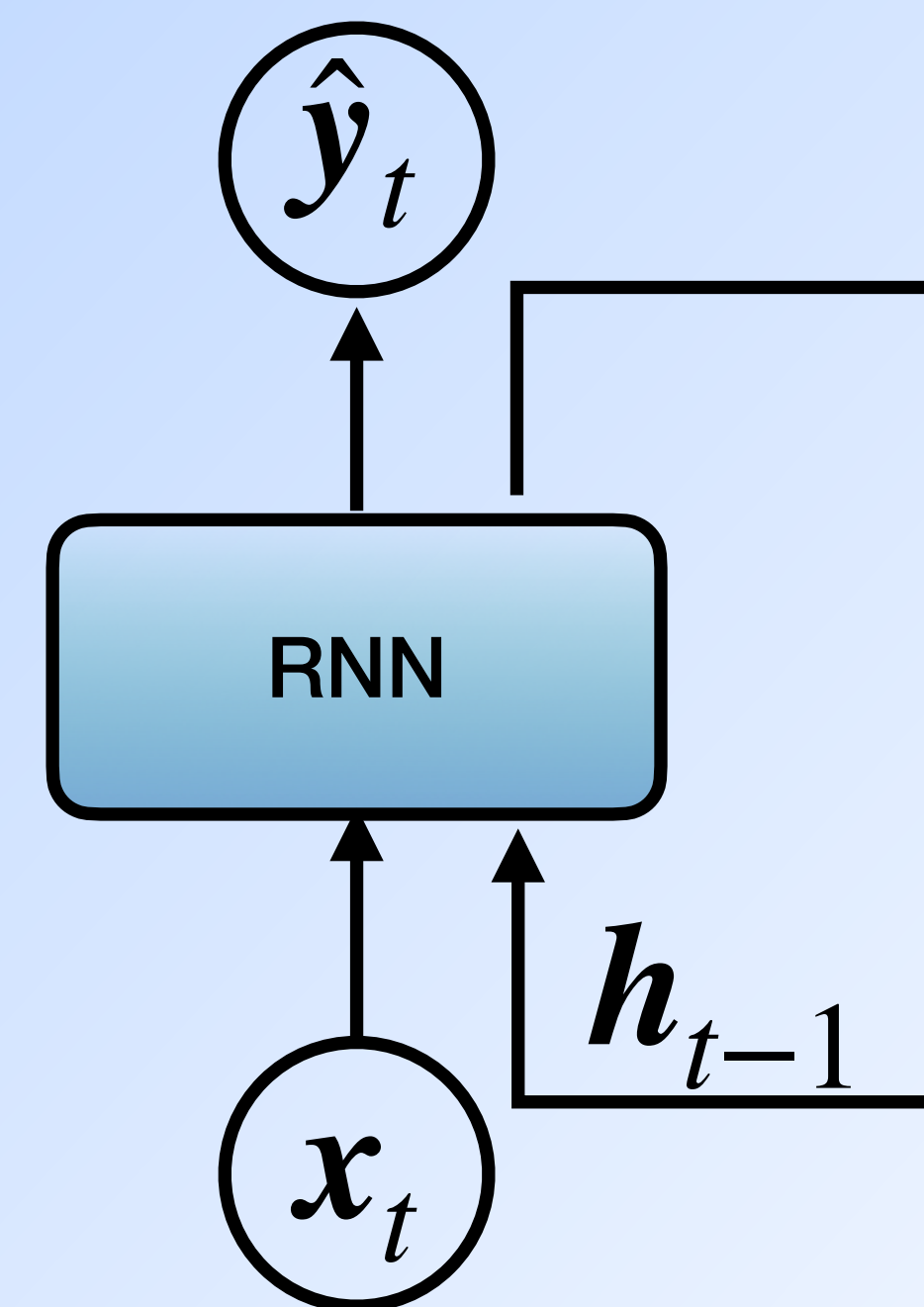
$$\mathbf{x}_{1:T} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T\}, \mathbf{x}_t \in \mathbb{R}^n$$

- A recurrent neural network maps the input to the output as follows:

$$\mathbf{h}_t = \phi_h(W_h \mathbf{h}_{t-1} + W_x \mathbf{x}_t + \mathbf{b}_h)$$

$$\hat{\mathbf{y}}_t = \phi_y(W_y \mathbf{h}_t + \mathbf{b}_y)$$

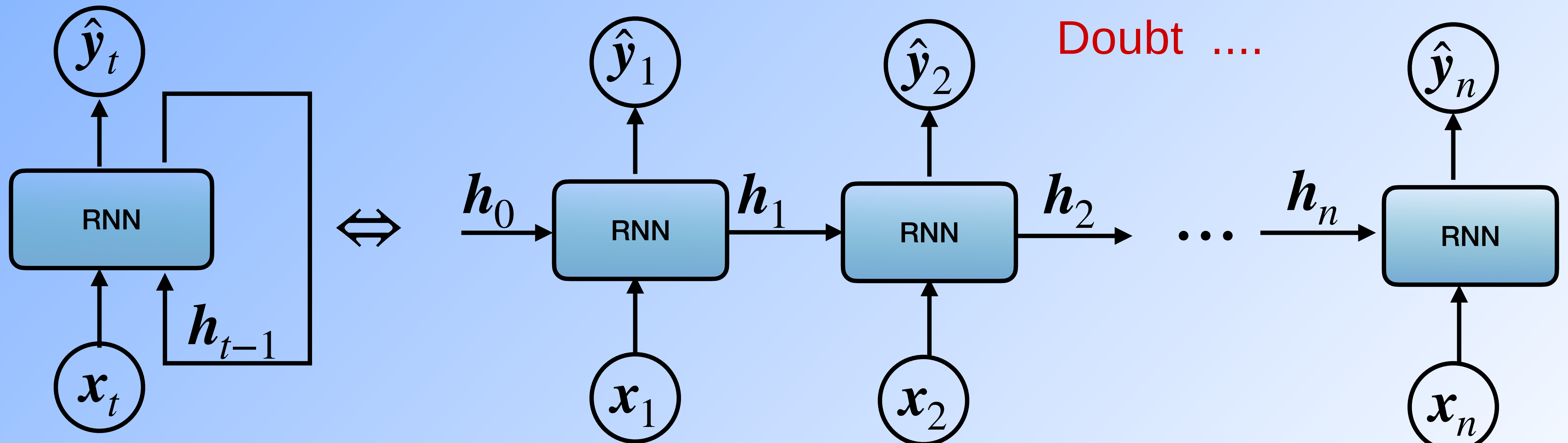
$$\mathbf{h}_0 = \mathbf{0} \text{ (zero initialization of the hidden state)}$$



one layer of RNN + o/p layer

Recurrent Neural Networks

- The RNN at time step t could be viewed as a neural network having inputs x_t and h_{t-1}



h_t and x_t can be of different dimensions.

x_n or x_T ?

these are not diff layers, here all these layers are sharing same parameters , its one layer only unfolded over time dimension

$\hat{y}_1$ may not be produced at each time step, it depends on the task suppose it is many to one task then it produces $\hat{y}_T$ at the last time step

Backpropagation Through Time

Let the cost function of RNN be given as follows:

$$J(\boldsymbol{\theta}) = \sum_{t=1}^T \ell(\mathbf{y}_t, \hat{\mathbf{y}}_t)$$

where $\boldsymbol{\theta} = \{W_h, W_x, W_y, \mathbf{b}_h, \mathbf{b}_x, \mathbf{b}_y\}$ are the network parameters.

$$\frac{\partial J(\boldsymbol{\theta})}{\partial W_h} = \sum_{t=1}^T \frac{\partial \ell(\mathbf{y}_t, \hat{\mathbf{y}}_t)}{\partial W_h}$$

$$\frac{\partial \ell(\mathbf{y}_t, \hat{\mathbf{y}}_t)}{\partial W_h} = \frac{\partial \ell(\mathbf{y}_t, \hat{\mathbf{y}}_t)}{\partial \hat{\mathbf{y}}_t} \cdot \frac{\partial \hat{\mathbf{y}}_t}{\partial \mathbf{h}_t} \cdot \frac{d\mathbf{h}_t}{dW_h}$$

Backpropagation Through Time

Since $\mathbf{h}_t$ depends on W_h and also on $\mathbf{h}_{t-1}, \mathbf{h}_{t-2}, \dots, \mathbf{h}_1$, each of which is also a function of W_h we can write:

$$\frac{d\mathbf{h}_t}{dW_h} = \frac{\partial \mathbf{h}_t}{\partial W_h} + \sum_{k=1}^{t-1} \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} \frac{\partial \mathbf{h}_k}{\partial W_h}$$

$$\begin{aligned} \text{So, } \frac{\partial \ell(\mathbf{y}_t, \hat{\mathbf{y}}_t)}{\partial W_h} &= \frac{\partial \ell(\mathbf{y}_t, \hat{\mathbf{y}}_t)}{\partial \hat{\mathbf{y}}_t} \cdot \frac{\partial \hat{\mathbf{y}}_t}{\partial \mathbf{h}_t} \cdot \left(\frac{\partial \mathbf{h}_t}{\partial W_h} + \sum_{k=1}^{t-1} \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} \frac{\partial \mathbf{h}_k}{\partial W_h} \right) \\ &= \frac{\partial \ell(\mathbf{y}_t, \hat{\mathbf{y}}_t)}{\partial \hat{\mathbf{y}}_t} \cdot \frac{\partial \hat{\mathbf{y}}_t}{\partial \mathbf{h}_t} \cdot \left(\frac{\partial \mathbf{h}_t}{\partial W_h} + \sum_{k=1}^{t-1} \prod_{j=k}^{t-1} \frac{\partial \mathbf{h}_{j+1}}{\partial \mathbf{h}_j} \frac{\partial \mathbf{h}_k}{\partial W_h} \right) \end{aligned}$$

In practice, the full BPTT as expressed in the above equation is truncated to compute gradients for M prior time steps.

Vanishing and Exploding Gradients in BPTT

$$\mathbf{h}_t = \phi_h(W_h \mathbf{h}_{t-1} + W_x \mathbf{x}_t + b_h)$$

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} = \text{diag}(\phi'_h(W_h \mathbf{h}_{t-1} + W_x \mathbf{x}_t + b_h)) W_h$$

$$\prod_{j=k}^{t-1} \frac{\partial \mathbf{h}_{j+1}}{\partial \mathbf{h}_j} = \prod_{j=k}^{t-1} \text{diag}(\phi'_h(W_h \mathbf{h}_j + W_x \mathbf{x}_{j+1} + b_h)) W_h$$

Since the gradient involves product of $t - k$ Jacobians, the gradient might vanish or explode depending on the magnitudes of the eigenvalues of the Jacobians $\frac{\partial \mathbf{h}_{j+1}}{\partial \mathbf{h}_j}$ when $t - k$ is large.

why eigenvalues ?

Long Short Term Memory (LSTM)

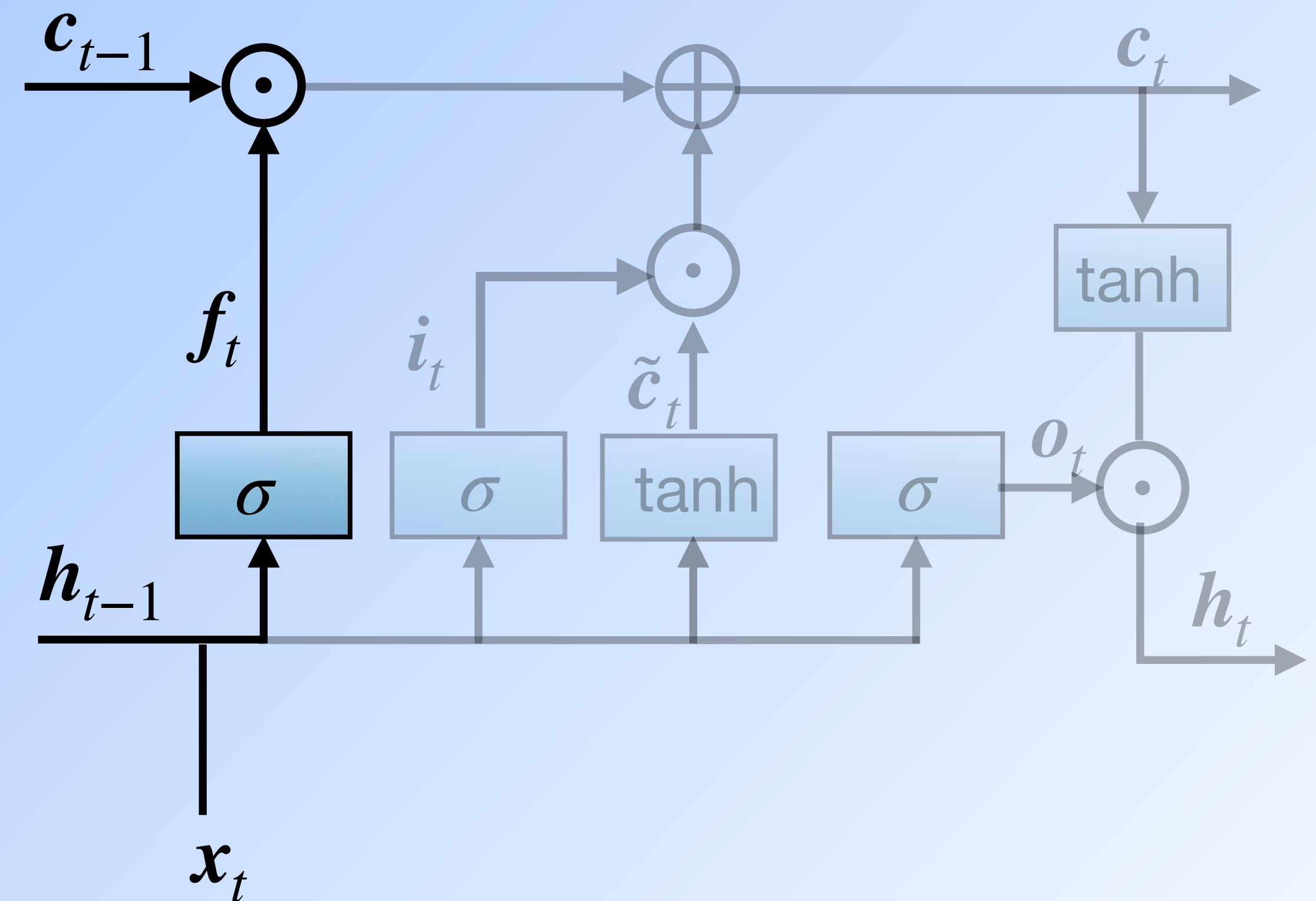
- As a consequence of vanishing and exploding gradients, vanilla RNNs are difficult to train and are unable to capture long term dependencies.
- LSTMs were introduced to address the vanishing/exploding gradient problem in vanilla RNNs, and learn long term dependencies.
- LSTMs (Hochreiter and Schmidhuber, 1997) employ a gating mechanism that effectively controls the flow of information into and out of its internal memory, called the cell state.
- A gate is a sigmoid layer followed by element wise multiplication.
- LSTM comprises the following gates:
 - Input gate
 - Forget gate
 - Output gate
- Allows information to be added or deleted from the cell state using the input and forget gates.

Long Short Term Memory (LSTM)

- The cell state vector $\mathbf{c}_t$ has the same dimension as the hidden state $\mathbf{h}_t$.
- *Forget gate*: the forget gate is used to perform a soft erasure of past information stored in the cell state that is no longer relevant:

$$\mathbf{f}_t = \sigma(W_f \mathbf{h}_{t-1} + U_f \mathbf{x}_t + \mathbf{b}_f)$$

if the output of the forget gate is close to 0, the corresponding information is forgotten.

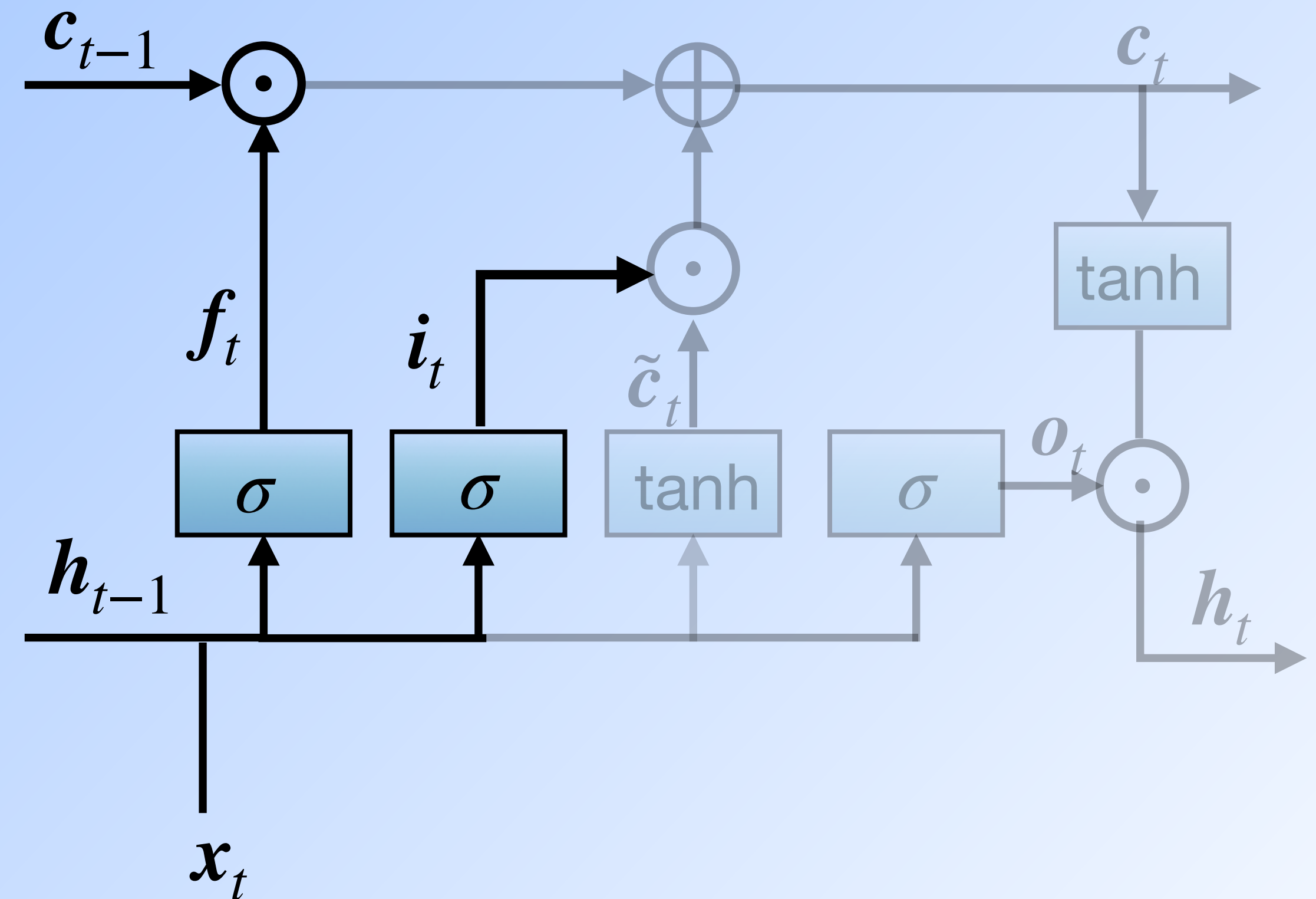


$\mathbf{x}_t$ and $\mathbf{h}_{t-1}$ are fed as input to the forget gate, o/p is $\mathbf{f}_t$ and $\mathbf{f}_t$ is multiplied element-wise with the current content of the cell state representation i.e. $\mathbf{c}_{t-1}$ by doing Hadamond product of $\mathbf{f}_t$ with $\mathbf{c}_{t-1}$

Long Short Term Memory (LSTM)

- Input gate*: the input gate is responsible for updating the cell state from the previous time step with relevant new information made available at the current time step.

$$i_t = \sigma(W_i h_{t-1} + U_i x_t + b_i)$$

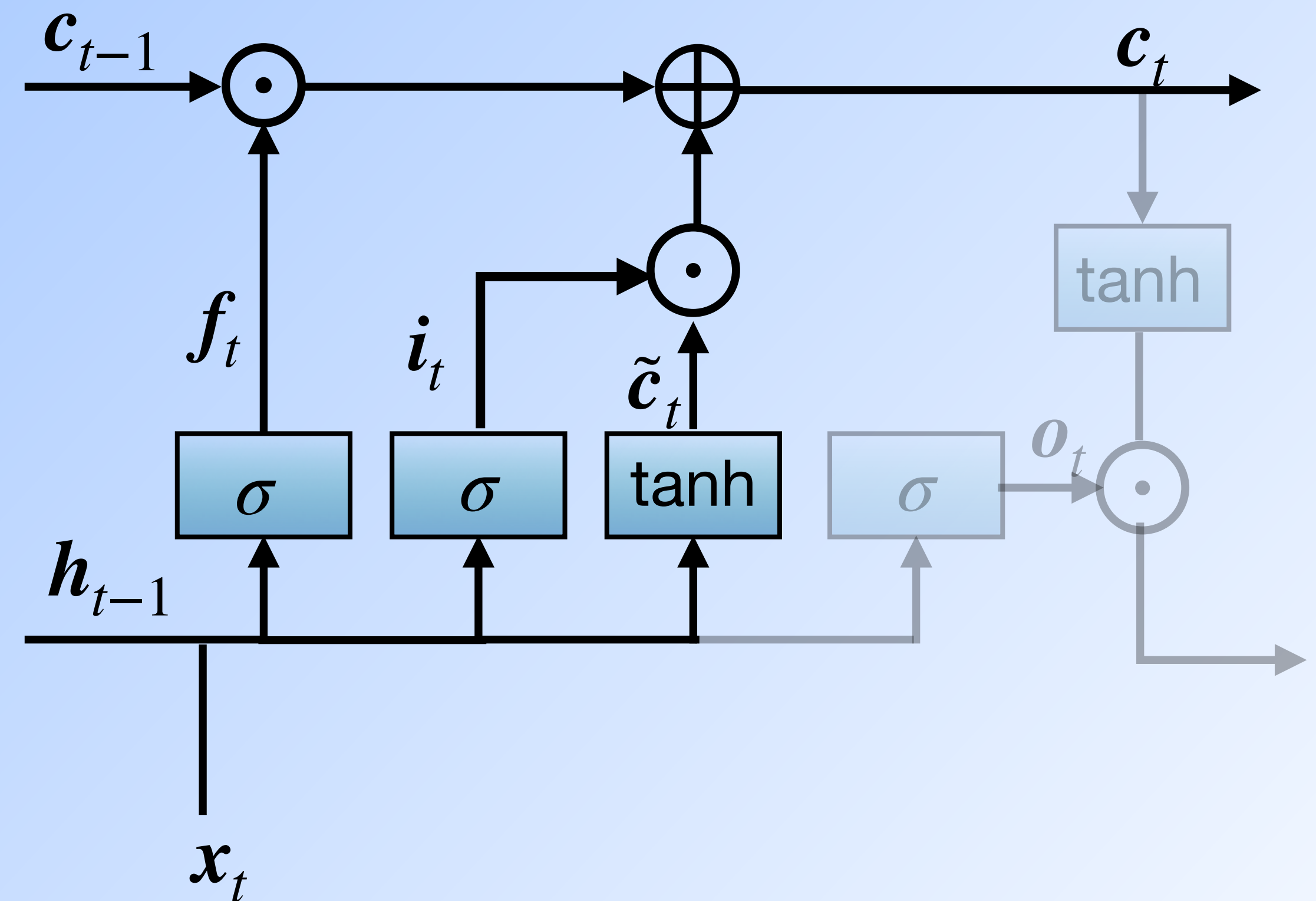


Long Short Term Memory (LSTM)

- The cell state vector $\mathbf{c}_t$ is updated by adding the relevant information from the current time step (as specified by the input gate) to the cell state at the previous time step after filtering the irrelevant information (as specified by the forget gate).

$$\tilde{\mathbf{c}}_t = \tanh(W_c \mathbf{h}_{t-1} + U_c \mathbf{x}_t + \mathbf{b}_c)$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$$



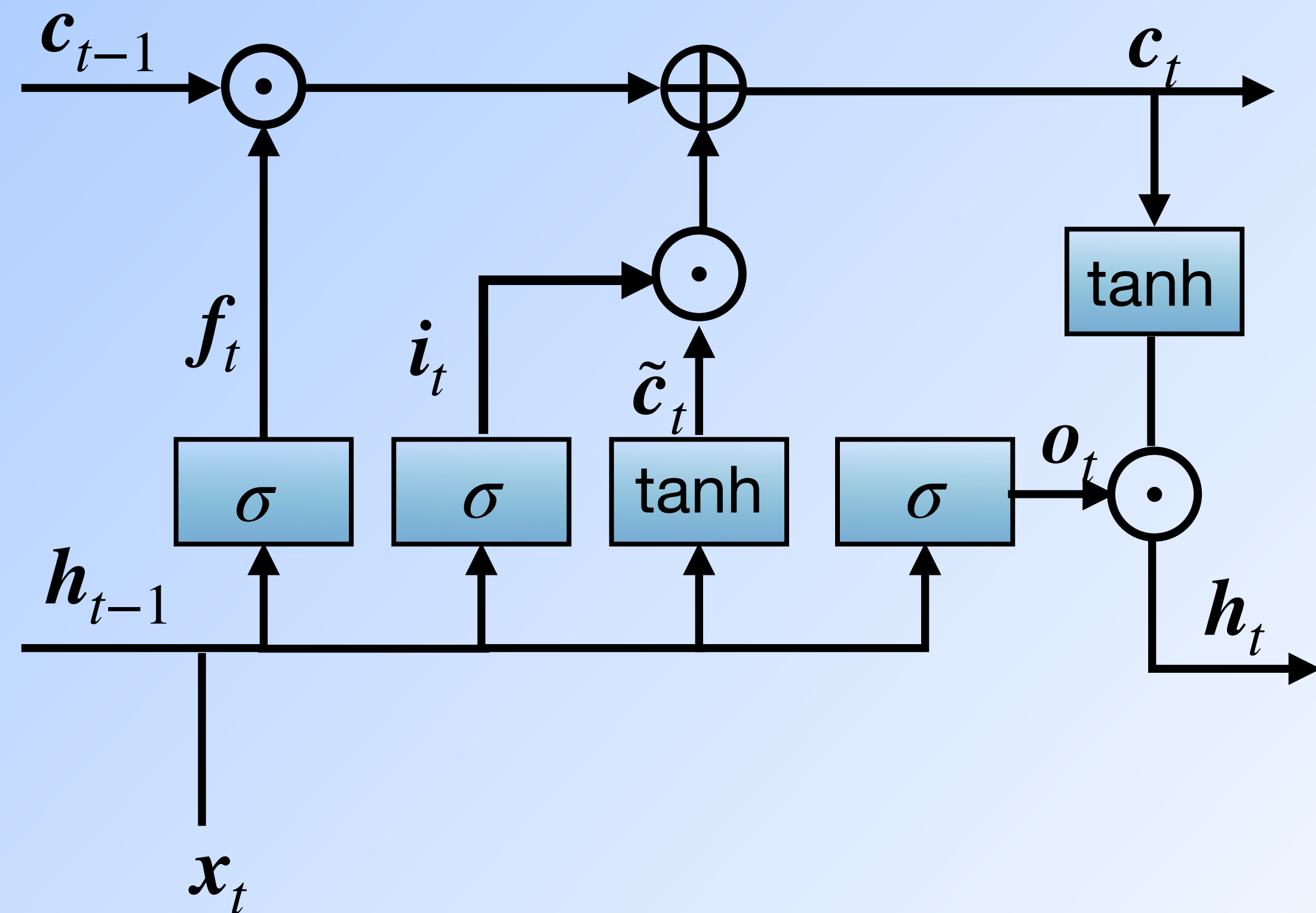
Long Short Term Memory (LSTM)

- Output gate*: the output gate extracts useful information from the current cell state to be used as output of the current time step.

$$\mathbf{o}_t = \sigma(W_o \mathbf{h}_{t-1} + U_o \mathbf{x}_t + \mathbf{b}_o)$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$$

Here, tanh activation is used to regulate the values of the hidden state to lie in the range $[-1, 1]$.



for cell state update, we use tanh so that values can get regulated in the range $[-1,1]$.
 for cell state we use tanh for having mechanism to increase or decrease the values as compared to previous cell state (add and subtract) but for o/p $\mathbf{h}_t$ we can use any bounded function either logistic or tanh

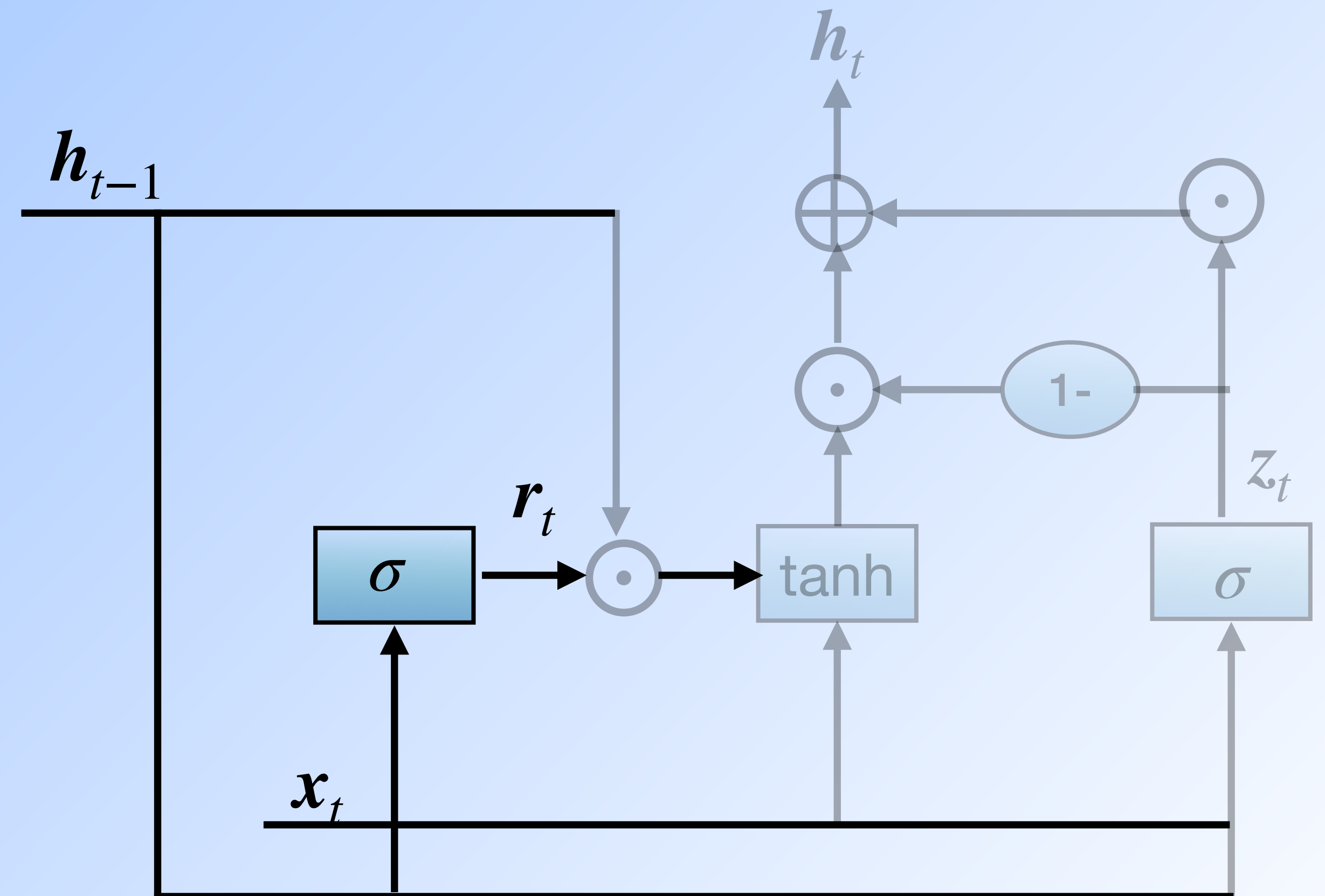
Long Short Term Memory (LSTM)

- The additive update rule for the cell state as well as the forgetting mechanism enabled by the forget gate allows LSTMs to learn long term dependencies, and empirically lessen the chance of vanishing gradients.
- However, this advantage comes at the cost of:
 - Increased computation and training times
 - Susceptibility to overfitting
 - Reduced interpretability

Gated Recurrent Unit (GRU)

- The Gated Recurrent Unit (GRU) ([Cho et al., 2014](#)) simplifies the LSTM architecture to reduce the number of parameters, thereby making it more computationally efficient and easier to train.
- Unlike LSTM, GRU uses just two gates, the reset gate and update gate, and does not maintain a separate cell state vector.
- Reset Gate: the reset gate determines how much of the previous hidden state information to forget:

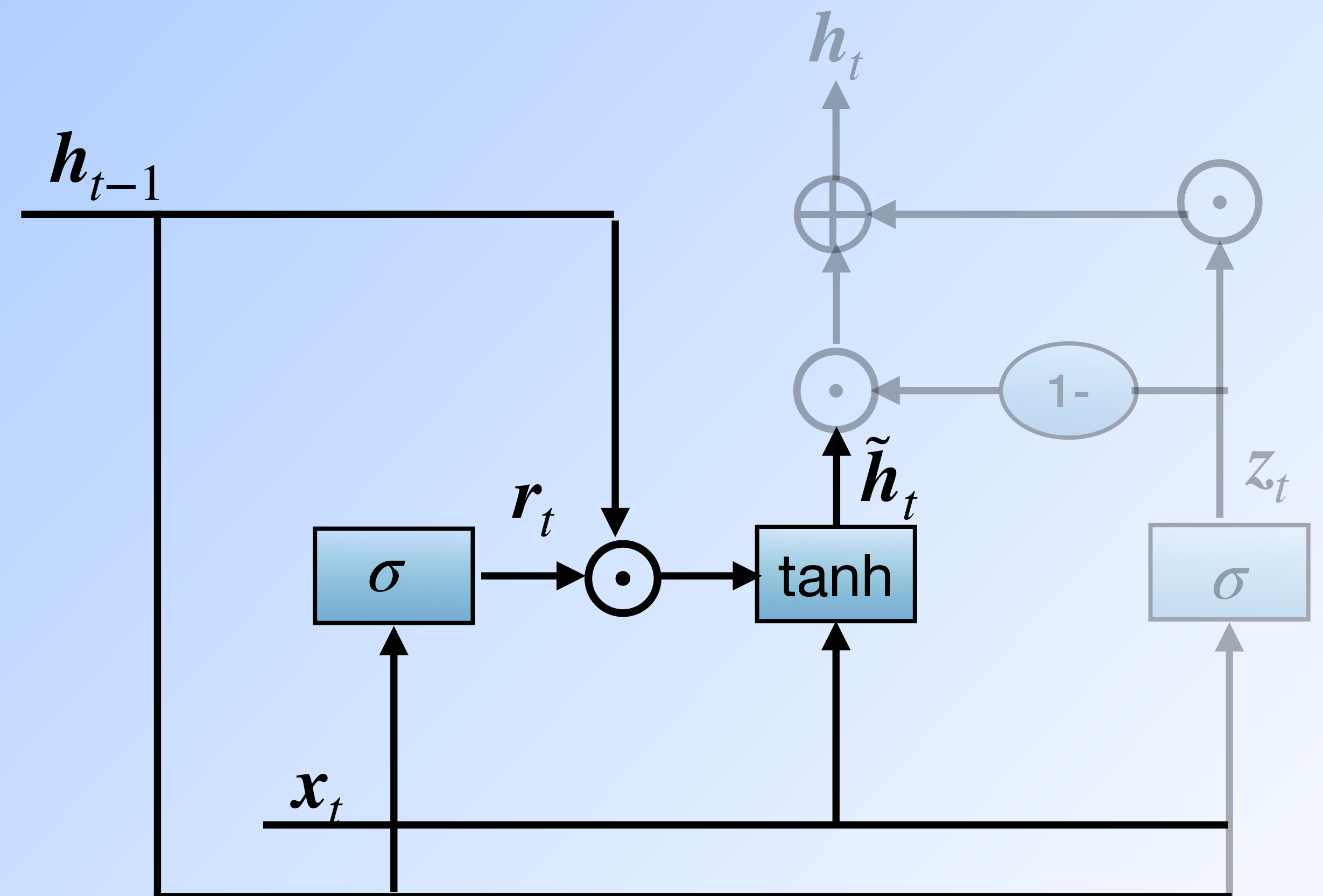
$$\mathbf{r}_t = \sigma(W_r \mathbf{x}_t + U_r \mathbf{h}_{t-1} + \mathbf{b}_r)$$



Gated Recurrent Unit (GRU)

- The candidate hidden state $\tilde{h}_t$ is computed by applying the reset gate output to the previous hidden state and combining the result with the current input as follows:

$$\tilde{h}_t = \tanh(W_h \mathbf{x}_t + U_h(\mathbf{r}_t \odot \mathbf{h}_{t-1}) + \mathbf{b}_h)$$

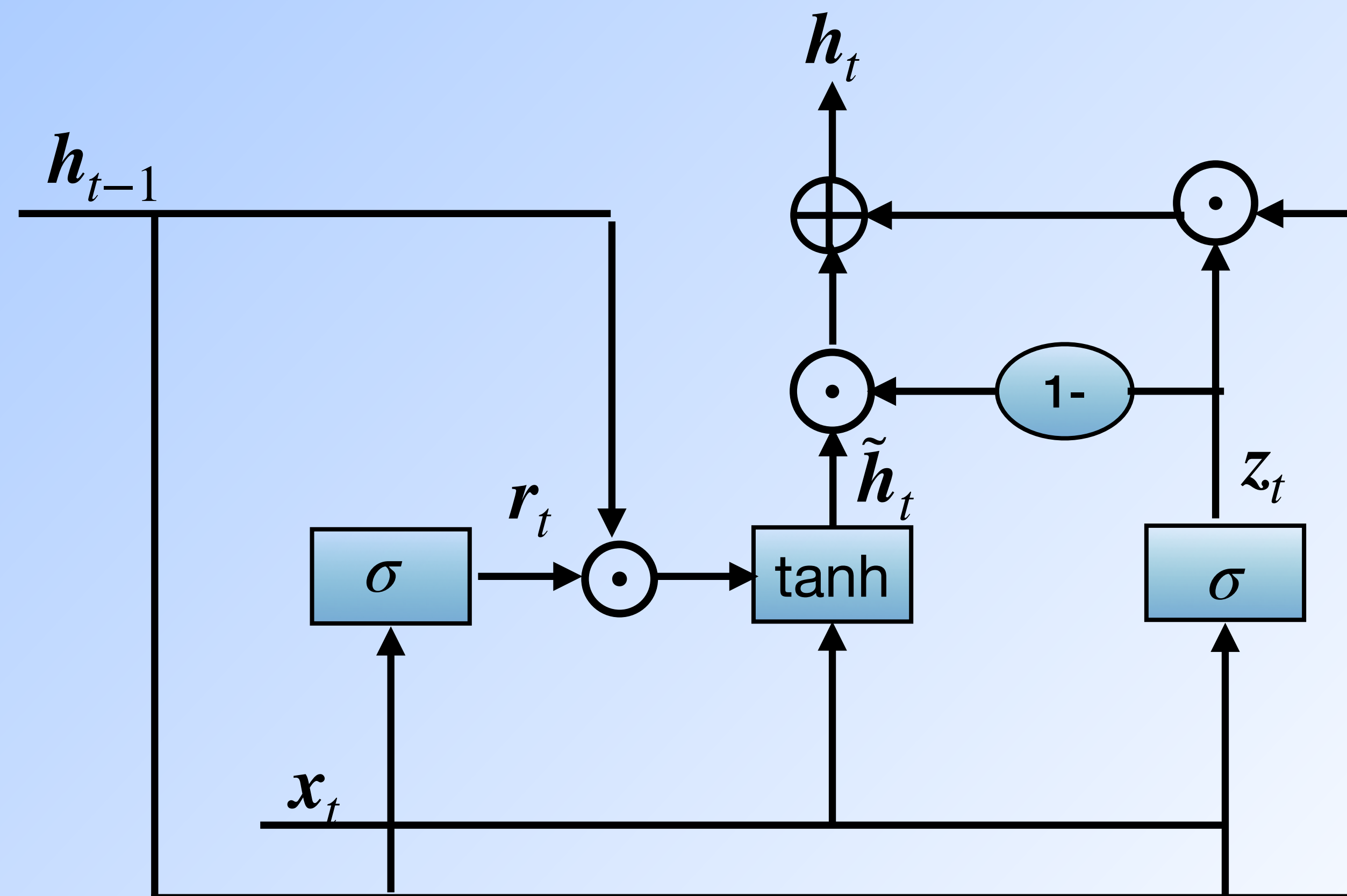


Gated Recurrent Unit (GRU)

- The update gate controls how much of the candidate hidden state is incorporated in the new hidden state as follows:

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z)$$

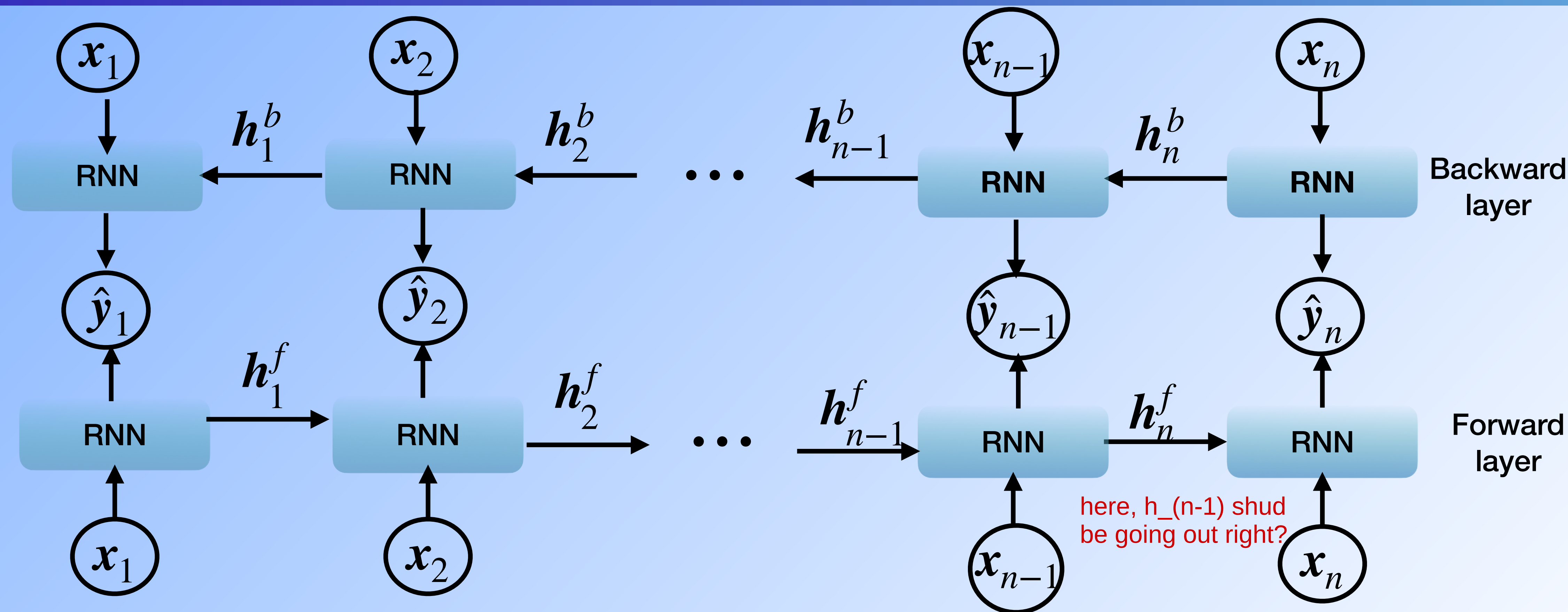
$$h_t = z_t \odot h_{t-1} + (1 - z_t) \odot \tilde{h}_t$$



Bidirectional RNNs

- The RNNs that we have looked at so far can only use the past context to make a prediction.
- However, there are problems (such as, action localization, machine translation), where it is useful to consider the future context as well.
- In such cases, the input sequence could be processed both in the forward and reverse order to make a prediction.
- Bidirectional RNNs process the input sequence in both normal and reverse orders and aggregates the outputs to make predictions.

Bidirectional RNNs



- The output of the forward and backward layers can be merged using various modes such as sum, averaging, concatenation etc.